CSE 4345: Big Data Analytics

Week 3, Part 1 — The Big Data Ecosystem: HDFS & the Distributed Storage Foundation

Department of Computer Science & Engineering
Premier University, Chittagong

Semester 2026

Course & Lecture Information

Lecture Identity

Course:	CSE 4345
Week / Part:	3 of 11 | Part 1 of 2
CLO:	CLO1
PLO:	PLO(a)

CLO1

"Understand big data characteristics"

Course Progression

Week 1: *What* is Big Data (3Vs)
Week 2: *Why* integration is hard
Week 3: *Where* data is stored (HDFS)
Week 4: *How* data is processed (MapReduce)

Part 1 Covers

- The architecture shift: RDBMS → distributed storage
- The core principle: move computation to data
- The Hadoop ecosystem stack (layered)
- HDFS: NameNode, DataNode, Secondary NameNode
- Block storage and rack-aware replication
- HDFS read & write paths
- High availability (Active / Standby NameNode)
- The small file problem
- Ivy League alignment
- Student assessment pool

Lecture Agenda

1. The Architecture Motivation
2. The Hadoop Ecosystem Stack
3. HDFS: Architecture & Design
4. HDFS: Replication, Read & Write Paths
5. Ivy League Alignment
6. Student Assessment Pool

The Architecture Motivation

Why a New Storage Architecture?

The 2003 Problem Statement

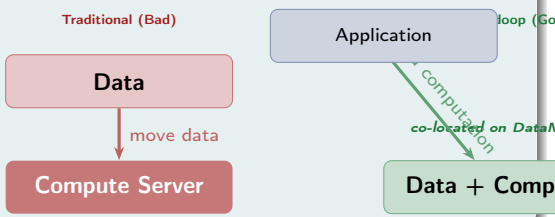
Google's web crawl produced petabytes of data that needed to be:

- **Stored** reliably across thousands of machines
- **Queried** for web indexing (sequential scans of entire datasets)
- **Fault-tolerant**: hardware failure expected daily at this scale
- **Cheap**: commodity PCs, not mainframes

What Existing Systems Could NOT Do

- **NFS / POSIX FS**: single server, no horizontal scale
- **RDBMS**: tuned for random access, not

The Paradigm Shift



The Fundamental Insight

Moving computation is cheaper than moving data. A 100 GB input file costs seconds to process locally; copying it over a network costs minutes.

The Core Principle: Data Locality

Definition: Data Locality

Running computation **on the same physical node** that holds the required data, avoiding network transfer.

Network transfer cost (2024 estimates):

Transfer medium	Throughput
Local disk read (SSD)	~3,000 MB/s
Local disk read (HDD)	~150 MB/s
Intra-rack network (1G)	~125 MB/s
Cross-rack network (1G)	~100 MB/s

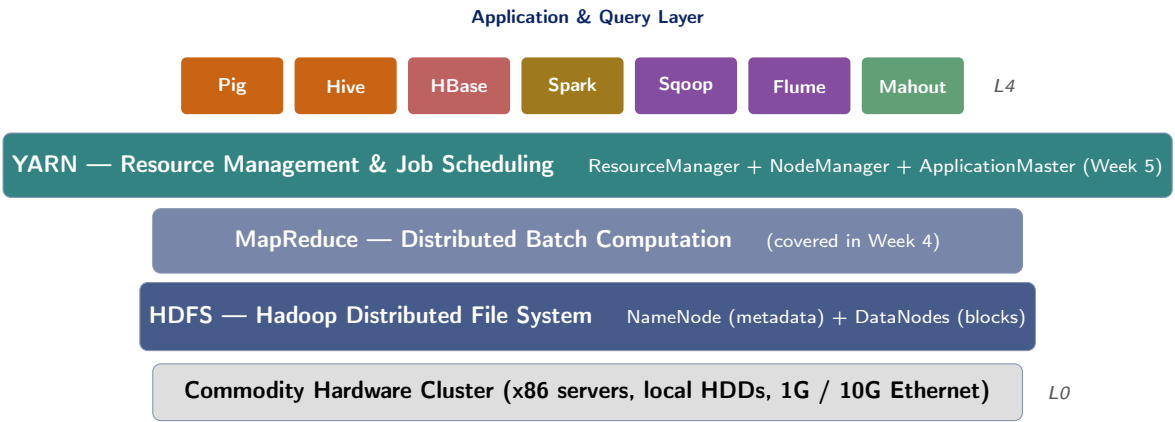
Three Levels of Locality (MapReduce / YARN)

1. **Data-local**: Task runs on the *same node* as the block
   - Best case: no network I/O
2. **Rack-local**: Task runs on a *different node in the same rack*
   - One hop through top-of-rack switch
3. **Off-rack**: Task runs on a *different rack*
   - Crosses the core network; worst case

YARN's scheduler tries level 1 first, falls back to level 2, then 3.

The Hadoop Ecosystem Stack

The Hadoop Ecosystem: Layered Architecture



Design Philosophy

Each layer is **independently replaceable**. YARN replaced MapReduce's resource manager in Hadoop

Ecosystem Components: Roles at a Glance

Component	Week	Role
HDFS	3	Distributed file system; stores data as blocks across DataNodes
MapReduce	4	Parallel batch computation model (Map + Shuffle + Reduce)
YARN	5	Cluster resource manager; schedules CPU/RAM across all frameworks
Pig	4	Dataflow scripting language (Pig Latin) for ETL pipelines
Hive	7	SQL-on-Hadoop; translates HiveQL to MapReduce / Spark jobs
HBase	4	NoSQL column-family store for low-latency random read/write
Spark	6	In-memory distributed computing; 10–100× faster than MapReduce

The Economics of Commodity Hardware

Google's Original Design Mandate (GFS, 2003)

"The system is built from many inexpensive commodity components that often fail. It must constantly monitor itself, detect, tolerate, and recover promptly from component failures."

Failure rates at scale:

Failure type	Rate (per year)
Individual disk failure	2–4%
Individual server failure	5–10%
Power supply failure	1–2%
Rack switch failure	0.5–1%
In a 1,000-node cluster	**50–100 failures/year**

Design Consequence

At 1,000+ nodes, hardware failure is not an **exception**—it is **routine**. HDFS must handle failures *transparently* without operator intervention or data loss.

Cost Comparison (2024 estimates)

	SAN Array	HDFS Cluster
Storage (1 PB)	\$500K+	\$50K
Redundancy	RAID	3× replication
Scalability	Vendor	Add nodes
Lock-in	High	None

HDFS achieves **10× cost reduction** while adding fault tolerance by design.

HDFS: Architecture & Design

HDFS Master / Worker Architecture

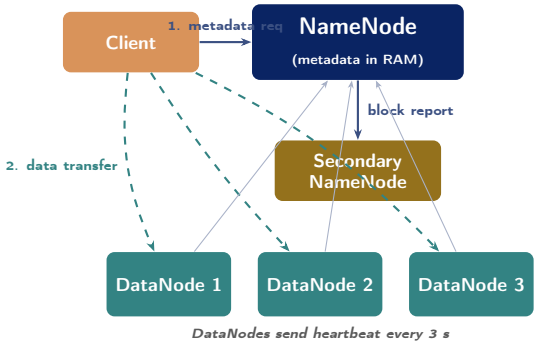
Three Core Components

- **NameNode** (1 active): the master. Stores all filesystem *metadata* in RAM.
- **DataNode** (*n* workers): the storage nodes. Store actual data *blocks* on local disks.
- **Secondary NameNode** (1): performs periodic metadata checkpointing. NOT a failover.

Two Distinct Planes

- **Metadata plane**: Client ↔ NameNode (file names, block locations, permissions)
- **Data plane**: Client ↔ DataNode (actual bytes read / written)

Separating these planes is a **critical design**

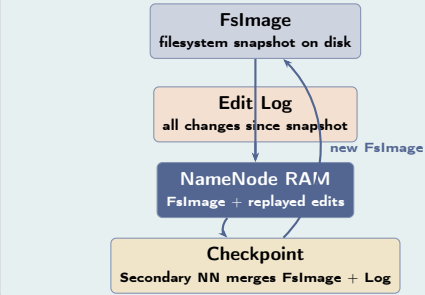


NameNode: The Brain of HDFS

What the NameNode Stores (in RAM)

- **Namespace tree**: complete directory hierarchy of all files and directories
- **File → block mapping**: for each file, which blocks it consists of, in order
- **Block → DataNode mapping**: where each block replica is currently stored (rebuilt from DataNode reports at startup)
- **Permissions**: POSIX-style owner, group, access rights

NameNode Metadata Persistence



Why RAM? Why Dangerous?

In-memory metadata = **microsecond lookups** (vs. disk-based = milliseconds). But: the NameNode is a **single point of failure**. If it crashes, the entire

Startup Sequence

NameNode loads **FsImage** from disk → replays all **Edit Log** entries → waits for DataNodes to report block locations → enters *safe mode* until

Secondary NameNode: The Critical Misconception

Common Exam Mistake

The Secondary NameNode is NOT a failover!

Its name is misleading. If the active NameNode crashes, the Secondary NameNode **cannot automatically take over**. The cluster goes down.

What It Actually Does: Checkpointing

1. Periodically fetches the current **FsImage** and **Edit Log** from NameNode
2. Merges them into a new, compact **FsImage**
3. Pushes the merged FsImage back to NameNode
4. Clears the Edit Log on NameNode

Why needed? Without checkpointing, the Edit Log grows indefinitely, making NameNode restart slower

Analogy: Bank Ledger

- **FsImage** = the bank's *balance sheet* (snapshot at end of month)
- **Edit Log** = the bank's *transaction journal* (every deposit/withdrawal this month)
- **Secondary NameNode** = the *accountant* who reconciles the journal into a new balance sheet at month-end, reducing the journal to zero
- Without the accountant, the journal grows without bound and takes longer to audit each time

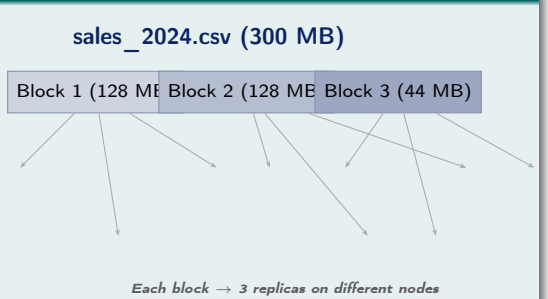
True Failover: HDFS HA (Hadoop 2.0+)

DataNode: Block Storage in Practice

What a DataNode Does

- Stores data as **flat files on local disk** (one file per HDFS block)
- Sends a **heartbeat** to NameNode every 3 seconds (liveness signal)
- Sends a **block report** to NameNode every 6 hours (full inventory of stored blocks)
- Serves **client read requests** directly (no NameNode involved in data transfer)
- Participates in **replication pipeline** for writes
- Runs **block scanner** to detect silent bit-rot (checksum verification)

Block Storage: How a 300 MB File is Stored



NameNode Detection of DataNode Failure

Block Size: Why 128 MB?

Traditional Filesystem Block Size: 4 KB

In a traditional OS (ext4, NTFS):

- 4 KB block = minimise *internal fragmentation* (wasted space within a block)
- Files are small (KB to MB range)
- **Random access** is common: seek to any byte quickly

HDFS Block Size: 128 MB (default, Hadoop 2.x+)

In HDFS:

- Files are **large** (GB to TB): a 10 GB file = 80 blocks at 128 MB
- Workloads are **sequential reads**: MapReduce

The Small File Problem

HDFS's Achilles' heel: if you store **millions of small files** (e.g., 1 KB each):

- Each file = at least 1 block = 1 metadata entry in NameNode
- 10 million files $\approx$ **10 million metadata objects** in RAM
- NameNode RAM exhaustion $\rightarrow$ cluster unusable
- Each task processes only 1 KB but incurs full task-startup overhead

Solutions:

- **HAR files**: bundle small files into a Hadoop Archive

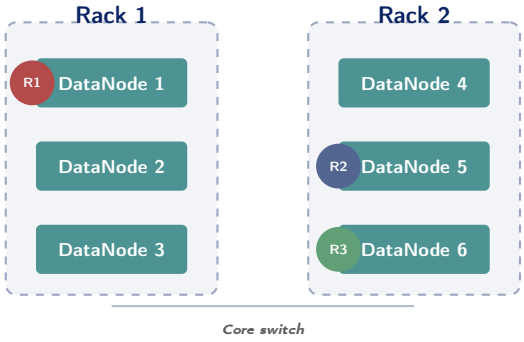
HDFS: Replication, Read & Write Paths

Rack-Aware Replication Strategy

Default Replication Factor: 3

HDFS places three replicas of each block using a **rack-aware** policy to tolerate *both* node failure and rack failure:

1. **Replica 1**: Same node as the writer client (data-local write)
2. **Replica 2**: A different node in a *different rack* (off-rack)
3. **Replica 3**: Another node in the *same rack as Replica 2* (for intra-rack bandwidth)



What Failures Are Tolerated?

- **Any 1 node fails**: 2 replicas remain; replication restored automatically

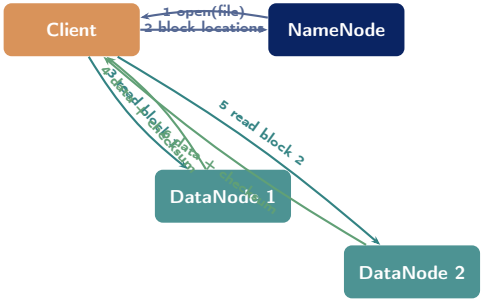
Configuring Replication

Default: `dfs.replication=3`. Change per-file:
`hdfs dfs -setrep 2 /path/file`

HDFS Read Path

Read Sequence (6 Steps)

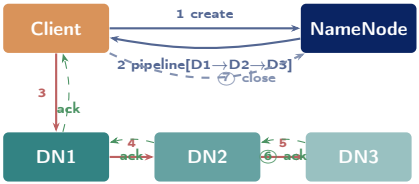
1. Client opens file via **DistributedFileSystem API**
2. **NameNode** returns ordered list of block locations (sorted by network proximity to client)
3. Client connects to the **closest DataNode** for Block 1 and reads it
4. Client reads Block 1; **checksum** verified on each chunk
5. Client moves to the next block → next-closest DataNode
6. Process repeats until all blocks read; client closes the stream



HDFS Write Path: Pipeline Replication

Write Sequence (7 Steps)

1. Client calls `create()` on NameNode
2. NameNode returns a **pipeline**: [DN1 → DN2 → DN3]
3. Client writes packets to **DN1 only**
4. DN1 forwards each packet to **DN2** (intra-rack)
5. DN2 forwards to **DN3** (cross-rack)
6. Acknowledgements flow **back through the pipeline**: DN3 → DN2 → DN1 → Client
7. After all blocks written, client calls `close()`; NameNode records the file as complete



Write Failure Handling

If a DataNode fails mid-pipeline, HDFS removes it,

HDFS in Practice: Key CLI Commands

Essential HDFS Shell Commands

```
# List files in HDFS directory
hdfs dfs -ls /user/data/

# Upload local file to HDFS
hdfs dfs -put sales.csv /user/data/

# Download file from HDFS
hdfs dfs -get /user/data/sales.csv ./

# Read file contents
hdfs dfs -cat /user/data/sales.csv

# Delete file
hdfs dfs -rm /user/data/old.csv

# Create directory
hdfs dfs -mkdir /user/data/2024/

# Check cluster health
hdfs fsck / -files -blocks -locations

# Cluster status report
```

Useful Admin Commands

```
# Change replication factor for a file
hdfs dfs -setrep 2 /data/file.csv

# Check NameNode storage info
hdfs dfs -df -h /

# Set quota on a directory
hdfs dfsadmin \
  -setSpaceQuota 500g \
  /user/team_data/

# Safe-mode operations (NN startup)
hdfs dfsadmin -safemode get
hdfs dfsadmin -safemode leave

# Balance data across DataNodes
hdfs balancer -threshold 10
```

HDFS vs POSIX

Ivy League Alignment

HDFS in Top University Curricula

MIT 6.830 — Database Systems (Madden, Stonebraker)

MIT uses HDFS/GFS as the canonical case study for **distributed file system design trade-offs**:

- **Strong consistency vs. availability**: NameNode is a CP system—consistent but unavailable during NN failure
- **SPOF analysis**: The single NameNode is the main architectural weakness; HA design is a distributed-systems engineering exercise
- **Metadata in RAM**: A deliberate trade-off—fast access in exchange for memory limits on namespace size ($\approx$1 billion files for 128 GB NameNode RAM)

Key exam question at MIT 6.830:
"How does HDFS enforce consistency without a distributed lock manager?" (Answer: **leases** on open files; NameNode is the sole authority on block locations.)

Harvard CS265 — Big Data Systems (Idreos)

Harvard's course emphasises the **storage hierarchy** impact on algorithm design:

- HDFS abstracts the disk and network into a **single storage namespace**
- Algorithms must be designed to **minimise the number of passes** over HDFS data (each full pass = petabytes of I/O)
- Harvard uses the concept of **I/O complexity** as a complexity measure, replacing time complexity for big data algorithms

Stanford CS246 (Leskovec)

Student Assessment Pool

Assessment—MCQ Set A

CLO1

Instructions

Select the single best answer. Correct answers **bolded** for instructor reference.

Q1. What is the **default block size** in HDFS (Hadoop 2.x and later)?

(a) 32 MB (b) 64 MB (c) **128 MB** (d) 256 MB

Q2. In HDFS, which component stores the **metadata** (file-to-block mappings, directory tree, permissions)?

(a) DataNode (b) **NameNode** (c) Secondary NameNode (d) ResourceManager

Instructor Notes

Q1: Hadoop 1.x default was 64 MB—students who read older textbooks may choose (b). The 128 MB default was introduced in Hadoop 2.0. **Q2:** Many students incorrectly select "Secondary NameNode"—reinforce that it performs checkpointing only, never stores primary metadata.

Q3. What is the **default replication factor** for blocks in HDFS?

(a) 1 (b) 2 (c) **3** (d) 5

Q4. The **Secondary NameNode** in HDFS primarily serves as:

(a) A hot standby that takes over automatically when the primary NameNode fails
(b) **A checkpoint service that periodically merges the Edit Log with the FsImage**
(c) A dedicated node for storing small file metadata to reduce NameNode load
(d) A load balancer that distributes client read requests across DataNodes

Instructor Notes

Q3: Replication factor 3 is the default but students should know it is *configurable* per-file. **Q4:** Option (a) is the most common wrong answer and represents the #1 misconception about HDFS. Spend class time on this distinction. Option (b) is the exact correct role.

Assessment—True / False *CLO1*

Instructions

State True or False and provide a **one-sentence justification** for full marks.

Statement	Answer
1. HDFS is optimised for low-latency random-access reads of small records.	**False**
2. The Google File System paper (Ghemawat et al., 2003) directly inspired the design of HDFS.	**True**
3. In HDFS, computation (MapReduce tasks) is moved to the nodes that hold the data, rather than moving data to a compute server.	**True**
4. If the active NameNode crashes, the Secondary NameNode automatically becomes the new NameNode and the cluster continues without interruption.	**False**

Assessment—Descriptive Questions *CLO1*

Instructions

Answer each question in **100–150 words**. Include a diagram where indicated.

D1. Explain the **master/worker architecture** of HDFS. Identify each component's role and describe what happens to **under-replicated blocks** when a DataNode fails permanently.

D2. Why does HDFS use a **128 MB block size** rather than the 4 KB blocks used by traditional filesystems? Name one type of workload that *benefits* from large blocks and one that is *harmed* by them (explain why in each case).

D3. **Draw and explain** the Hadoop ecosystem stack. For each of the four layers (HDFS, MapReduce, YARN, Applications), state its role and give one concrete component or tool that belongs to that layer.

Assessment—Analytical Questions *CLO1*

Instructions

Answer in **200–300 words** with step-by-step reasoning. Marks awarded for correct process, not just conclusion.

A1. **Replication Recovery Trace:**
A DataNode (DN4) that stores the **only** copies of blocks B7, B8, and B9 crashes permanently (it was hosting the 2nd replica of B7 and 3rd replica of B8, but the *only* replica of B9—the other DataNodes holding B9 also failed earlier).
- What does the NameNode do for B7 and B8 (under-replicated)?
- What is the situation for B9, and what does this mean for data that relied on it?
- How does the rack-aware replication policy determine *which* DataNodes receive the new replicas of B7 and B8?

A2. **HDFS vs. NFS:**
A university wants to store 50 TB of genomics research data that will be analysed by Spark jobs (sequential scans) but also queried interactively by individual researchers (random access to specific gene records). Compare HDFS and NFS on four dimensions (scale, fault tolerance, access pattern, cost) and recommend the appropriate system. Could a hybrid approach work? Justify.

Textbook & Reading References

Primary Textbooks for Week 3

- **[TW]** Tom White. *Hadoop: The Definitive Guide*, O'Reilly, 4th ed., 2015. **Chapters 1, 3**
  - Ch. 1: Meet Hadoop (motivation, history) · Ch. 3: HDFS (architecture, read/write, HA)
- **[SA]** Acharya & Chellappan. *Big Data and Analytics*, Wiley, 2015. **Chapter 3**
- **[TE]** Erl, Khattak & Buhler. *Big Data Fundamentals*, Prentice Hall, 2016. **Chapter 8**

Supplementary (Covered in Week 3, Part 2 — Seminal Paper)

- Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). **The Google File System.** *ACM SOSP*, pp. 29–43. **[PRIMARY SEMINAL PAPER]**
- Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). **The Hadoop Distributed File System.** *IEEE MSST. [How the GFS design was adapted for open-source Hadoop]*

Strongly Recommended Online Resource

HDFS Architecture Guide (Apache official documentation):
https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

— Sanjay Ghemawat, Google File System (2003)

Next Lecture: Week 3, Part 2

Seminal Paper: Ghemawat, Gobioff & Leung (2003) — *"The Google File System"*

Design principles · Chunk management · Consistency model · Master recovery
Case Study: Yahoo!'s 10,000-node Hadoop deployment (2008)

Reading before Part 2: [TW] Ch. 1 & 3 · GFS paper (see reference slide)